



Ataíde Gualberto, Nalanda Victoria, Matheus  
Tavares, Matheus Yuri, Mirelle Alves

# VISÃO GERAL

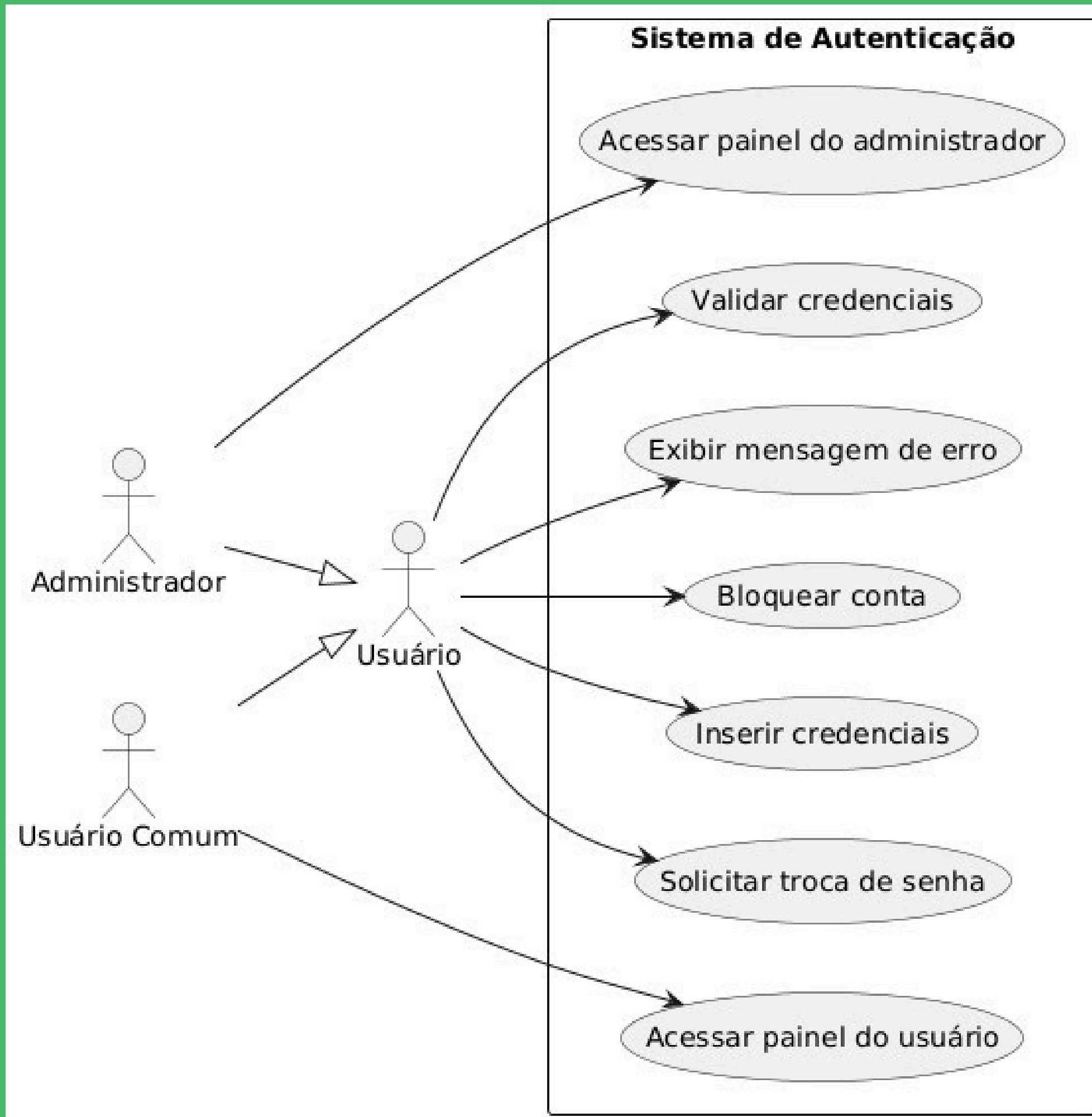
- Cadastro e gerenciamento de usuários e veículos;
- Controle de manutenção e despesas;
- Sistema de alertas inteligentes;
- Histórico detalhado e geração de relatórios;
- Compartilhamento seguro de veículos.

# DIAGRAMAS

# CASO DE USO

- Atores principais:
  - Proprietário do veículo e Usuário convidado;
- Funcionalidades do proprietário:
  - Gerenciar veículo, manutenção, despesas, alertas e compartilhamento;
- Funcionalidades do convidado:
  - Visualizar ou editar conforme permissão;
- Base para fluxos dos diagramas de atividade.

# CASO DE USO



# ATIVIDADE

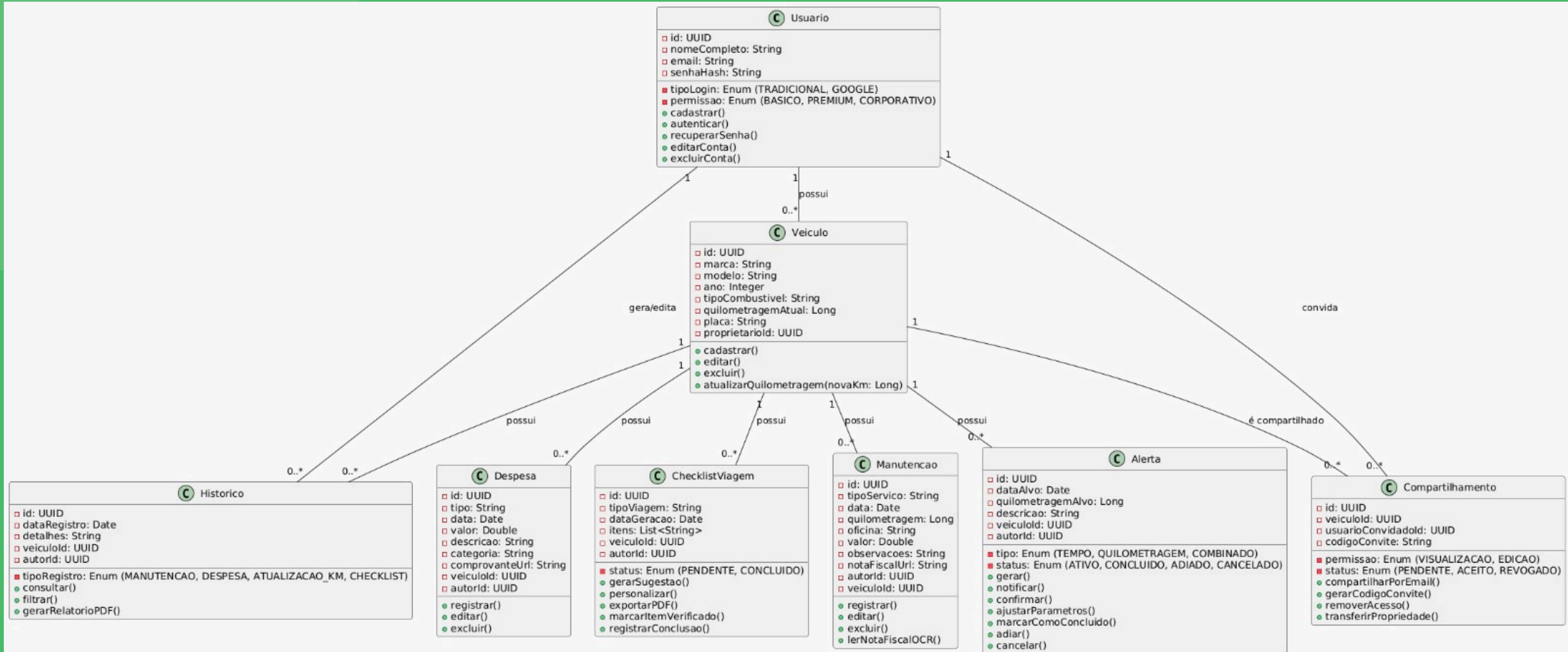
- Detalhamento do fluxo do compartilhamento de veículo;
- Decisões e verificações: método escolhido, cadastro do convidado;
- Sequência lógica das ações do sistema;
- Conecta ao diagrama de sequência para interações detalhadas.



# CLASSE

- Classes principais:
  - Usuário, Veículo, Manutenção, Alerta, Despesa, Histórico, Checklist, Compartilhamento;
- Atributos e métodos para manipulação dos dados;
- Relações entre classes:
  - Usuário → Veículo
  - Veículo → Manutenção, Alerta, etc.
- Base para estrutura de banco de dados e programação orientada a objetos.

# CLASSE

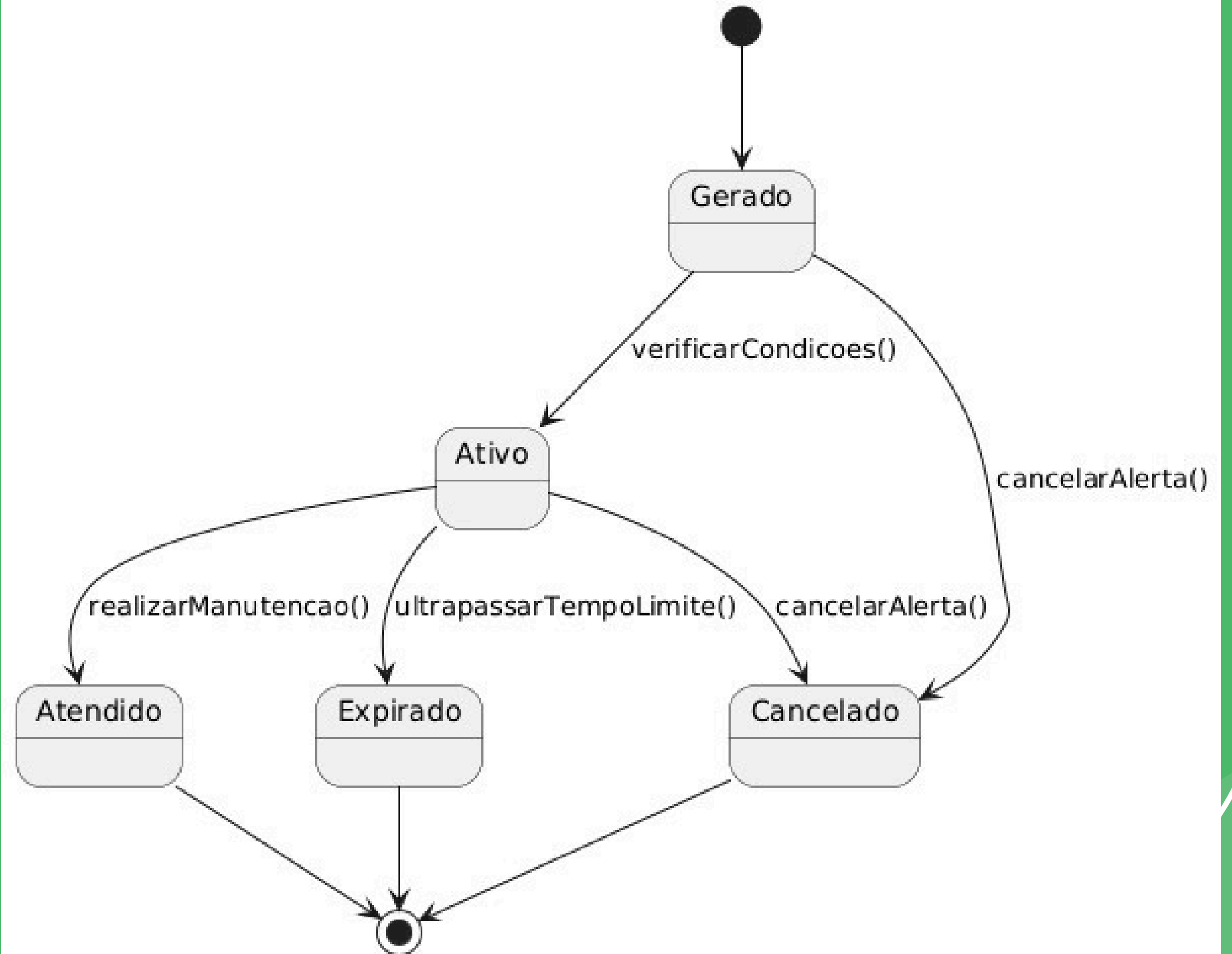


# ESTADO

- Ciclo de vida dos alertas de manutenção:
  - Gerado → Ativo → Atendido / Expirado / Cancelado;
- Representa mudanças de estado conforme ações e tempo;
- Automatiza notificações e atualizações no sistema;
- Modela comportamento dinâmico das entidades.

# ESTADO

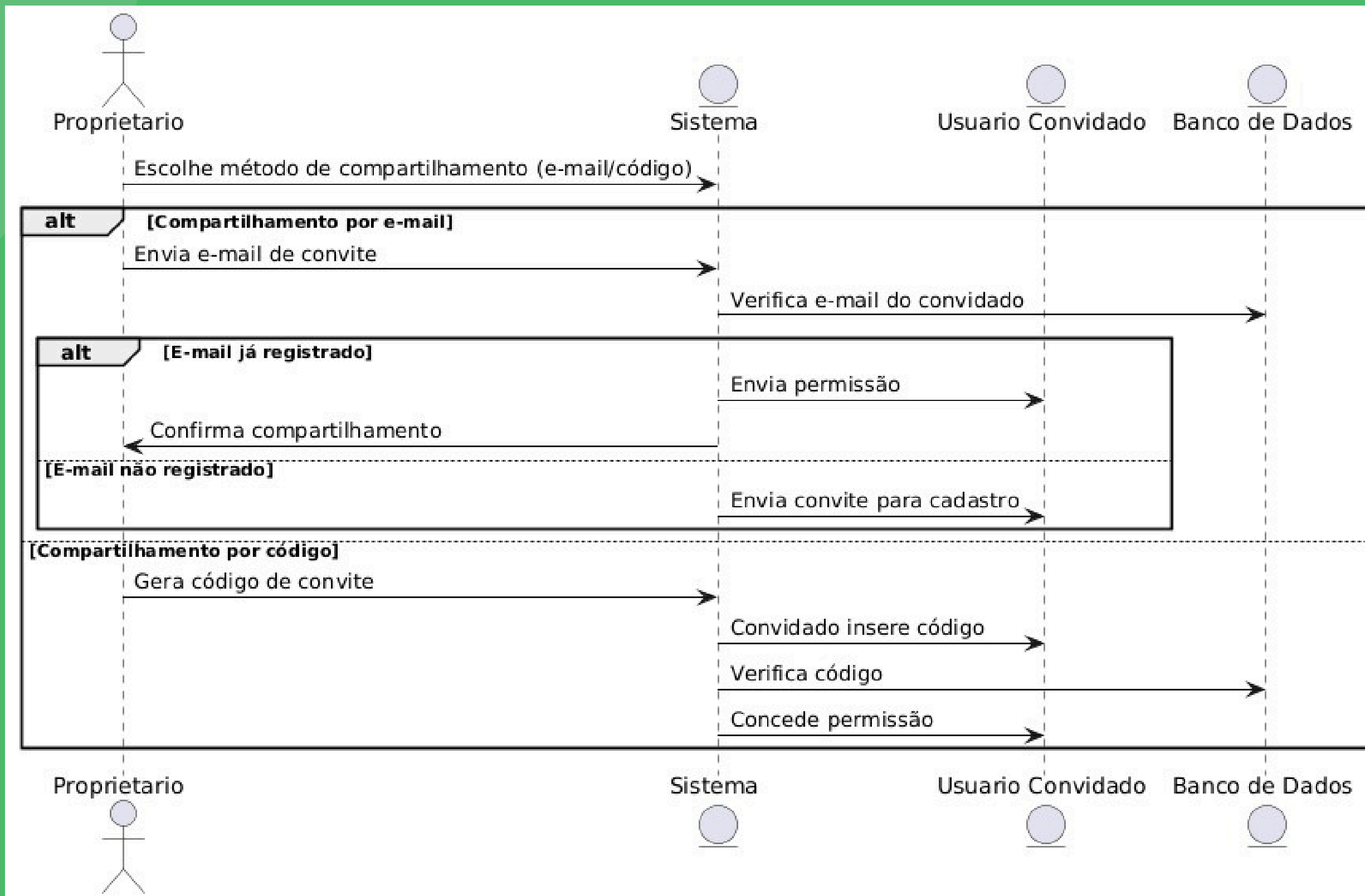
Diagrama de Estados - Alerta de Manutenção



# SEQUÊNCIA

- Interações temporais entre: Proprietário, Sistema, Banco de Dados, Usuário Convidado;
- Fluxo do compartilhamento de veículo;
  - Mensagens enviadas para validação e concessão de permissões;
  - Complementa diagramas de atividade e classe para implementação.

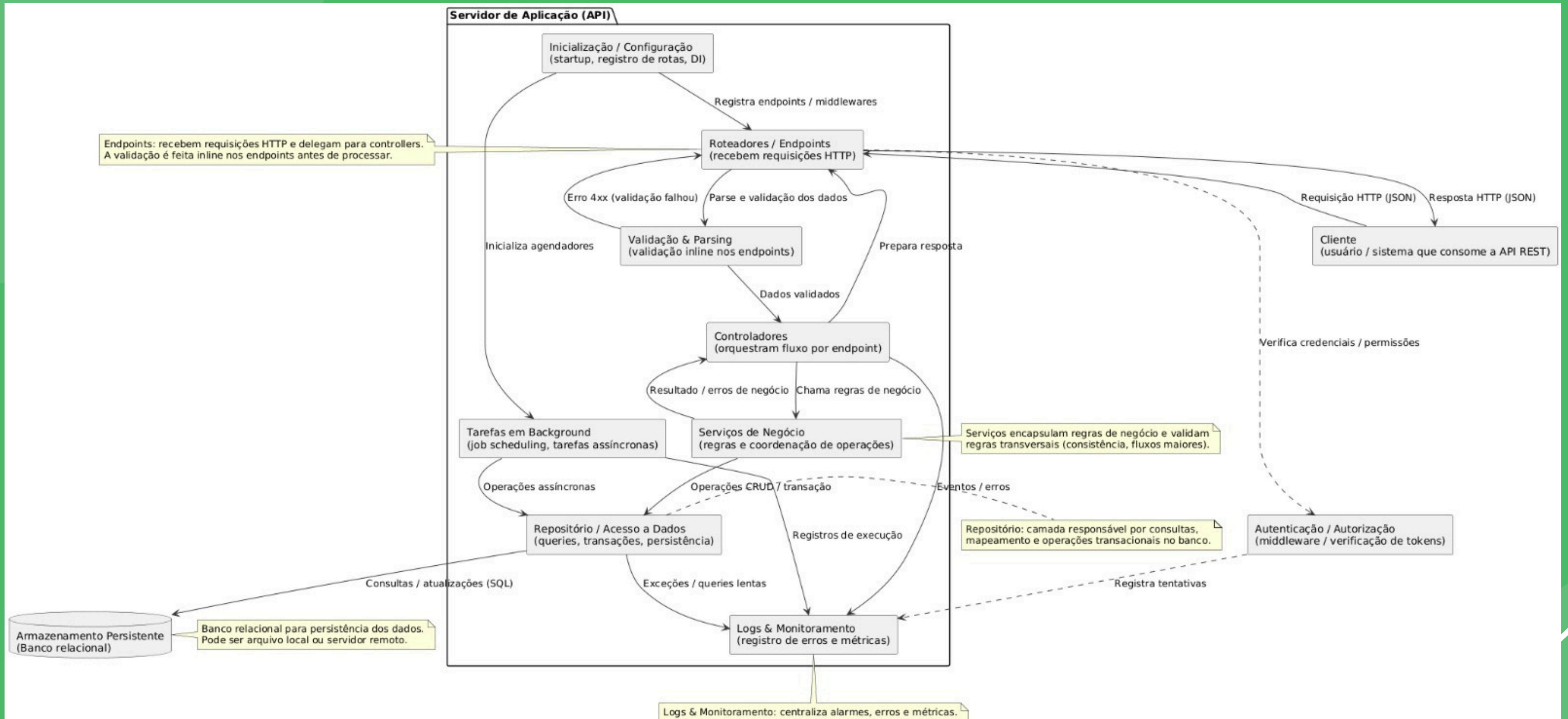
# SEQUÊNCIA



# ARQUITETURA

- Organização em camadas:
  - Apresentação
  - Negócio
  - Dados
- Responsabilidades claras para cada camada;
- Uso de padrão MVC para separação de responsabilidades;
- Facilita manutenção, escalabilidade e colaboração.

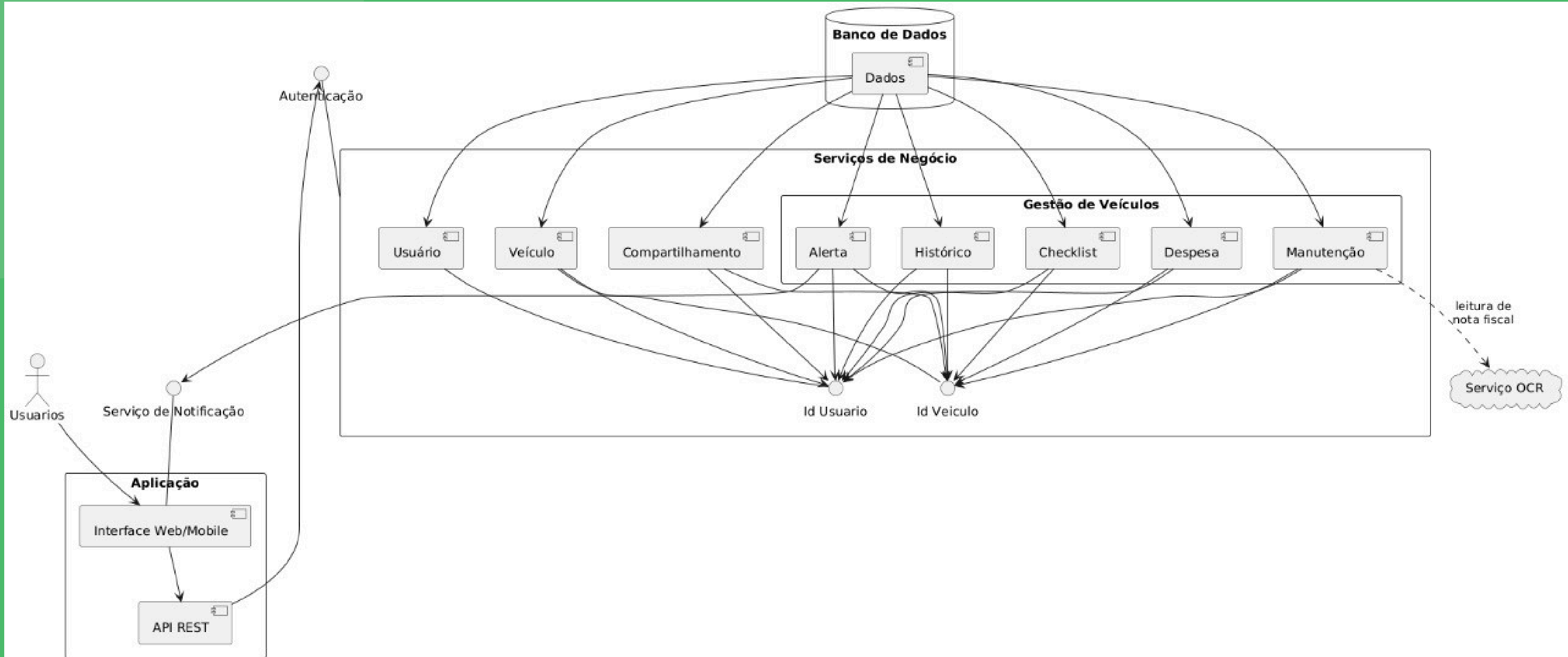
# ARQUITETURA



# COMPONENTES

- Componentes principais: Gerenciamento de usuários, Veículos, Manutenção, Alertas, Despesas e Compartilhamento;
- Modularidade e interfaces bem definidas;
- Facilita manutenção e testes.

# COMPONENTES



# CONCLUSÃO

- O Revisaí foi modelado como uma solução robusta para gerenciamento veicular, priorizando usabilidade, organização de dados e compartilhamento seguro.;
- A arquitetura definida, aliada a diagramas bem estruturados, fornece uma base sólida para um desenvolvimento ágil, modular e escalável.;

# CONCLUSÃO

- Os próximos passos incluem implementação das funcionalidades principais, integração com APIs externas, realização de testes unitários e de integração, além de refinamentos contínuos baseados em feedback técnico..

# Link do Github

[https://github.com/MatheusTavares-dev/2025.1\\_AOO\\_AV\\_REVISAI](https://github.com/MatheusTavares-dev/2025.1_AOO_AV_REVISAI)

# Aberto para dúvidas e sugestões!

